



Laboratorio di Sistemi Operativi

PRESENTATO A: ALESSANDRA ROSSI

PRESENTATO DA: ALESSANDRO MARTINIELLO, CRISTINA MARANO

MATRICOLA: N86004508, N86004511

Indice

1. Introduzione	3
2. Architettura di Rete e Gestione della Concorrenza.....	4
2.1 Gestione della Scalabilità Orizzontale	4
3. Protocollo di Comunicazione: Design e Semantica.....	5
3.1 Scelta del Formato Testuale	5
4. Logica di Gioco e Astrazione (tris_game)	6
4.1 Scelte di implementazione della Logica:.....	6
5. Gestione dello Stato e Robustezza delle Sessioni	7
5.1 Strutture Dati Centralizzate:	7
5.2 Gestione delle disconnessioni:	7
6. Implementazione del Client	8
7. Containerizzazione e Deployment	9
7.1 Dockerfile:.....	9
7.2 Docker Compose:.....	9
8. Conclusioni e Scalabilità.....	10
8.1 Valutazione delle Scelte:	10
8.2 Evoluzione del Sistema:	10

1. Introduzione

Questo progetto fornisce un'implementazione del classico gioco del Tris (Tic-Tac-Toe) giocabile in modalità multiplayer su rete. Il progetto è composto da un Server centrale che gestisce le partite e le interazioni tra i giocatori, e da un Client che permette agli utenti di connettersi al server, creare o unirsi a partite e giocare. L'intera applicazione è containerizzata utilizzando Docker e Docker Compose per facilitare il setup e il deployment.

2. Architettura di Rete e Gestione della Concorrenza

La scelta fondamentale nell'implementazione del server è stata l'utilizzo della system call `select()` per il multiplexing degli I/O, anziché l'uso di un approccio multi-threaded (uno per client).

Motivazioni della scelta di `select()`:

- **Efficienza delle Risorse:**

In un gioco a turni come il Tris, i client passano la maggior parte del tempo in attesa dell'input dell'utente. L'uso di `select()` permette a un singolo processo server di gestire fino a `MAX_CLIENT` (impostato a 256) senza il sovraccarico di memoria derivante dalla creazione di stack separati per ogni thread.

- **Semplicità di Sincronizzazione:**

Poiché tutto viene gestito in un unico thread principale, non è necessario l'uso di mutex o semafori per proteggere le strutture dati globali come l'array `games` o `clients`, eliminando il rischio di race conditions durante l'aggiornamento dello stato delle partite.

- **Reattività:**

Il server è in grado di rispondere immediatamente non appena un socket (sia esso il master per nuove connessioni o un client esistente) presenta dati pronti per la lettura.

2.1 Gestione della Scalabilità Orizzontale

Nonostante il limite attuale di 256 client e 128 partite simultanee, l'architettura è progettata per la scalabilità. L'uso di variabili d'ambiente per host e porta permette di replicare il servizio dietro un load balancer senza modificare una singola riga di codice C.

3. Protocollo di Comunicazione: Design e Semantica

Il protocollo è di tipo testuale, orientato ai comandi, facilitando il debug e la scalabilità.

Comandi implementati:

- **create:**

inizializza una nuova istanza di gioco nel server.

- **join <id>:**

Permette a un utente di richiedere l'accesso a una partita specifica

- **move <riga> <colonna>:**

Trasmette le coordinate della mossa.

- **accept / reject:**

Meccanismo di handshake per permettere al creatore della partita di scegliere il proprio avversario.

Questa separazione dei comandi permette una gestione granulare degli stati del giocatore (PLAYER_CONNECTED, PLAYER_IN_GAME, PLAYER_WAITING_ACCEPT).

3.1 Scelta del Formato Testuale

Si è optato per un protocollo "Human-Readable" invece di uno binario per diverse ragioni strategiche:

- **Flessibilità del Parsing:**

I comandi come create, join <id> e move <r> <c> permettono una validazione immediata tramite stringhe, facilitando l'estensione del protocollo con nuovi comandi senza rompere la retrocompatibilità.

- **Handshake e Sicurezza Logica:**

L'introduzione dei comandi accept/reject non è solo funzionale, ma serve a implementare un livello di "consenso" tra gli utenti, gestendo lo stato PLAYER_WAITING_ACCEPT per evitare che un giocatore sia forzato in una partita indesiderata.

4. Logica di Gioco e Astrazione (tris_game)

La logica del Tris è stata isolata in un modulo dedicato (tris_game.c / tris_game.h).

4.1 Scelte di implementazione della Logica:

- **Indipendenza dalla Rete:**

Le funzioni make_move, check_winner e print_board operano esclusivamente sulla struttura TrisGame, ignorando l'esistenza dei socket.

Questo rende la logica di gioco facilmente testabile separatamente dal server

- **Rappresentazione del Tabellone:**

È stata utilizzata una matrice 3x3 di un tipo enumerativo Cell (EMPTY, X, O). Questa scelta garantisce chiarezza nel codice.

- **Gestione del Turno:**

Il turno è gestito tramite un intero (0 per X, 1 per O).

Viene alternato con l'operazione `game->turn = 1 - game->turn` dopo ogni mossa.

5. Gestione dello Stato e Robustezza delle Sessioni

5.1 Strutture Dati Centralizzate:

Il server mantiene due strutture dati principali:

- **Client Array:**

Traccia il file descriptor, lo stato attuale e l'ID della partita associata a ogni utente.

- **Game Array:**

Contiene i file descriptor dei due sfidanti, lo stato della partita (GAME_NEW, GAME_WAITING_FOR_PLAYER, GAME_IN_PROGRESS, GAME_ENDED.) e l'istanza della logica di gioco.

5.2 Gestione delle disconnessioni:

Una scelta implementativa critica riguarda la robustezza: se un giocatore si disconnette, la funzione **remove_client** non solo chiude il socket, ma pulisce anche la partita associata tramite **remove_client_from_game**, notificando l'avversario e liberando lo slot per nuovi utenti.

6. Implementazione del Client

Il client è progettato per essere il più “snello” possibile (Thin Client).

È volutamente privato di logica decisionale per tre motivi:

- **Veridicità dello Stato:** Il server è l'unica "Source of Truth". Inviando stringhe pre-formattate (incluso il tabellone), si annulla la possibilità che un client mostri uno stato diverso da quello reale del server.
- **Semplicità di Aggiornamento:** Modificando l'estetica del tabellone sul server, tutti i client vedono il cambiamento istantaneamente senza dover aggiornare il software locale.
- **Interfaccia Non-Bloccante:** L'uso di `select()` anche lato client assicura che il programma rimanga reattivo agli eventi di rete mentre l'utente sta digitando una mossa.

7. Containerizzazione e Deployment

L'intero ecosistema è configurato per essere eseguito in ambienti isolati tramite Docker.

7.1 Dockerfile:

- **Immagine Base:**

È stata scelta **ubuntu:22.04** per garantire un ambiente di compilazione GCC standard e affidabile.

- **Compilazione Multi-Stage (Simulata):**

Il Dockerfile copia i sorgenti e compila sia il server che il client all'interno del container, garantendo che i binari siano compatibili con l'ambiente di esecuzione.

- **Dependency Management:**

Vengono installati strumenti di rete (net-tools, iputils-ping) per facilitare le diagnosi di rete tra container.

7.2 Docker Compose:

- **Networking:**

Viene definita una rete bridge personalizzata `game_network`. Questo permette al client di connettersi al server utilizzando il nome del servizio (server) come hostname, risolto automaticamente dal DNS interno di Docker.

8. Conclusioni e Scalabilità

L'implementazione ha dimostrato che un'architettura Single-Threaded basata su `select()` è la scelta ottimale per un gioco a turni, garantendo stabilità ed efficienza.

8.1 Valutazione delle Scelte:

- **Efficienza:**

Il multiplexing dell'I/O riduce drasticamente il footprint di memoria rispetto a un approccio multi-threaded

- **Integrità:**

La gestione in un unico thread principale elimina alla radice il rischio di race conditions nell'aggiornamento dello stato delle partite

- **Flessibilità:**

La separazione tra logica di gioco (`tris_game`) e rete permette di testare ed evolvere i moduli in modo indipendente

- **Centralizzazione:**

Il modello "Thin Client" assicura che il server rimanga l'unica fonte di verità, garantendo la sincronizzazione perfetta del tabellone per tutti gli utenti.

8.2 Evoluzione del Sistema:

Grazie all'uso di variabili d'ambiente e alla containerizzazione con Docker, il sistema è intrinsecamente pronto per il deployment in cloud. La scelta di un protocollo testuale e di una rete bridge personalizzata rende l'infrastruttura facilmente scalabile e configurabile per ambienti di rete complessi senza modifiche al codice sorgente.